

Mathematical Logic

Propositional logic and SAT solver

Satisfiability,
Conjunctive/disjunctive normal forms, Puzzles

Why logic

- Practical demand
 - ✓ Traditional testing is not enough for complex systems nowadays. Complementary use of formal method is desired.
 - ✓ Not only to say a system is correct, but also witness to be correct is required.
 - ✓ Human makes mistakes!
- Support by formal method
 - ✓ Recent progress of verification tools/libraries (improvement of both interface and implementation)
 - ✓ Progress of hardware (Limited memory)
 - ✓ The biggest advantage is, tool interaction makes you aware of your mistakes!

Propositional logic

- **Definition.** Atomic propositions : $A, B, C, \dots$
 - ✓ either *True* (1) or *False* (0) will be substituted
- **Definition.** Logical connectives :
 $\wedge$ (and), $\vee$ (or), $\neg$ (not), $\supset$ (imply)
- **Definition.** A proposition is,
 - ✓ Either an atomic proposition,
 - ✓ (Constants T and $\perp$), or
 - ✓ A pair of propositions combined with a logical connective.

How to prove automatically?

~ Two methods : syntax v.s. semantics ~

- Syntax: deduction rules
 - ✓ $A, A \supset B \Rightarrow B$ (modus ponens)
 - ✓ $(A \wedge B) \vee C \Rightarrow (A \vee C) \wedge (B \vee C)$
 - ✓ $(A \vee B) \wedge C \Rightarrow (A \wedge C) \vee (B \wedge C)$
 - ✓ $(A \supset B) \Rightarrow \neg A \vee B$

$A \wedge B$ and
 $B \wedge A$ are
different.

Equivalent
(completeness)

- Semantics: model (truth table)
 - ✓ $A \supset B$
 - ✓ $\neg A \vee B$

For First-order logic, semantics becomes complicated.

Model (Semantics)

- **Def. (Truth) Valuation** is a substitution (of boolean value, i.e., true/false) for each atomic proposition.
- For each logical connective, valuation is propagated as:

A	B	$\neg A$	$A \wedge B$	$A \vee B$	$A \supset B$
1	1	0	1	1	1
1	0	0	0	1	0
0	1	1	0	1	1
0	0	1	0	0	1

Validity

- **Def.** A proposition is **valid**, if it is always true for each valuation.
- **Exercise:** Are they valid?
 - ✓ $A \supset A$
 - ✓ $A \vee \neg A$
 - ✓ $((A \supset B) \supset A) \supset A$ (Pierce's law)

Satisfiability

- **Def.** A proposition is
 - ✓ **Satisfiable** $\Leftrightarrow$ there exists truth valuation such that the proposition becomes true.
 - ✓ **Unsatisfiable** $\Leftrightarrow$ there are no true valuation such that the proposition becomes true (i.e., always false for any substitutions on propositional variables).
- **Def.** A pair of propositions ψ , ϕ are **logically equivalent** if, for each valuation, truth values of ψ and ϕ coincide.
- Remark: ψ is unsatisfiable iff $\neg\psi$ is valid.

Conjunctive/Disjunctive Normal Forms (CNF / DNF)

Normal form

- **Def. Conjunctive normal form, CNF**
 - ✓ Clauses (= sequences of literals connected by $\vee$) are connected by $\wedge$.
 - ✓ $(A_1 \vee \neg A_2 \vee A_3) \wedge (B_1 \vee B_2 \vee B_3) \wedge (\neg C_1 \vee C_2)$
- **Def. Disjunctive normal form, DNF**
 - ✓ Sequences of literals connected by $\wedge$ are connected by $\vee$.
 - ✓ $(A_1 \wedge \neg A_2 \wedge A_3) \vee (B_1 \wedge B_2 \wedge B_3) \vee (\neg C_1 \wedge C_2)$
- Each proposition can be converted to CNF and DNF

Conversion to normal forms

- Implication elimination

$$\checkmark A \supset B \Leftrightarrow \neg A \vee B$$

- Negation distribution

$$\checkmark \neg(A \wedge B) \Leftrightarrow \neg A \vee \neg B$$

$$\checkmark \neg(A \vee B) \Leftrightarrow \neg A \wedge \neg B$$

- Double negation elimination

$$\checkmark \neg(\neg A) \Leftrightarrow A$$

Conversion to normal forms, contd.

- Distributive laws ($\wedge, \vee$)
 - ✓ $A \wedge (B \vee C) \Leftrightarrow (A \wedge B) \vee (A \wedge C)$
 - ✓ $A \vee (B \wedge C) \Leftrightarrow (A \vee B) \wedge (A \vee C)$
- Commutative / associative laws
 - ✓ $A \wedge B \Leftrightarrow B \wedge A$, $A \wedge (B \wedge C) \Leftrightarrow (A \wedge B) \wedge C$
 - ✓ $A \vee B \Leftrightarrow B \vee A$, $A \vee (B \vee C) \Leftrightarrow (A \vee B) \vee C$
- Idempotent
 - ✓ $A \wedge A \Leftrightarrow A$ $A \wedge \neg A \Leftrightarrow \text{false}$ $A \wedge \text{false} \Leftrightarrow \text{false}$
 $A \wedge \text{true} \Leftrightarrow A$
 - ✓ $A \vee A \Leftrightarrow A$ $A \vee \neg A \Leftrightarrow \text{true}$ $A \vee \text{false} \Leftrightarrow A$ $A \vee \text{true} \Leftrightarrow \text{true}$

Recursive operations to obtain normal forms

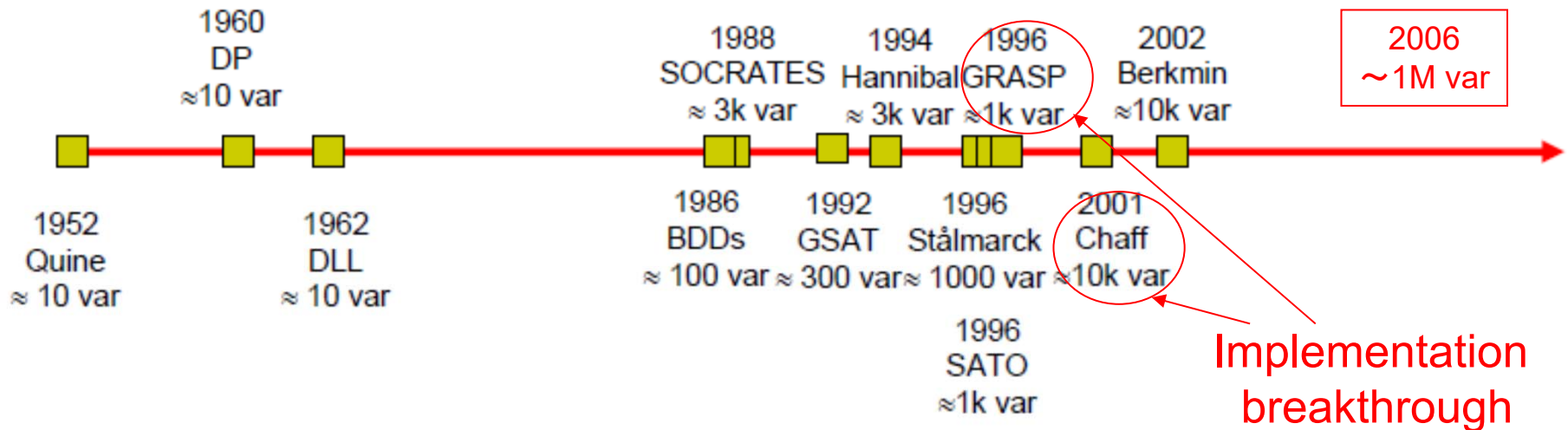
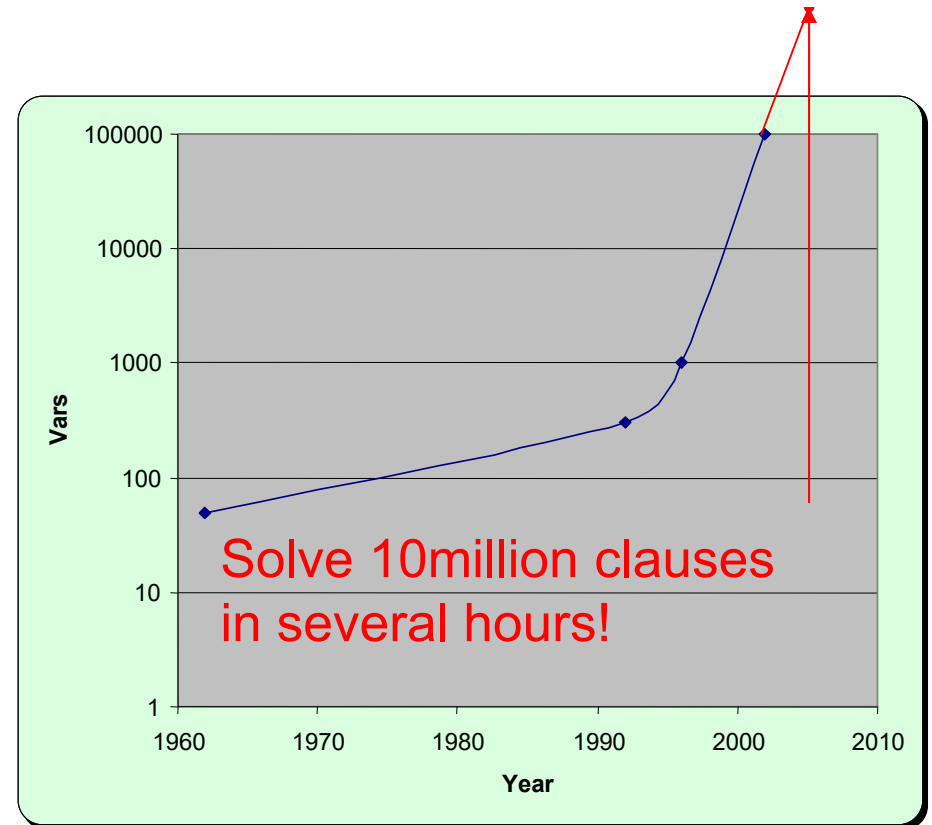
- **Th.** Each proposition A has a logically equivalent conjunctive (resp. disjunctive) norm form.
- **Proof.** First, replace all implications $B \supset C$ by $\neg B \vee C$. Then, by induction on structure of a proposition.
 - ✓ A is an atomic proposition. OK
 - ✓ $A = \neg B$. Let B' be a DNF of B . $\neg B'$ can easily lead a CNF of A .
 - ✓ $A = B \wedge C$. Let B' and C' be CNFs of B and C , resp.
 - ✓ $A = B \vee C$. Let $B'_1 \wedge \dots \wedge B'_m$ and $C'_1 \wedge \dots \wedge C'_n$ be CNFs of B and C , resp. $(B'_1 \vee C'_1) \wedge \dots \wedge (B'_m \vee C'_n)$ is a CNF of A .

Note : A CNF by this construction can be exponentially large.

Implementation of SAT solver
~Decide whether given CNF is satisfiable~

History of SAT

- SAT competition (2002~)
www.satcompetition.org/
- Prime factor (decimal)
 - ✓ 15-16 digits, *ok*
 - ✓ 34 digits, *no*



MiniSat Page - Microsoft Internet Explorer

ファイル(F) 編集(E) 表示(V) お気に入り(A) ツール(T) ヘルプ(H)

戻る 進む 検索 お気に入り メディア

アドレス(D) <http://www.cs.chalmers.se/Cs/Research/FormalMethods/MiniSat/> 移動

Google 検索

The MiniSat Page

by Niklas Eén, Niklas Sörensson

Main

MiniSat

MiniSat+

SatELite

Papers

Authors

Links

INTRODUCTION

MINISAT is a minimalistic, open-source SAT solver, developed to help researchers and developers alike to get started on SAT. It is released under the MIT licence, and is currently used in a number of projects (see "Links"). On this page you will find binaries, sources, documentation and projects related to MINISAT, including the Pseudo-boolean solver MINISAT+ and the CNF minimizer/preprocessor SATELITE. Together with SATELITE, MINISAT was



ページが表示されました

インターネット

Pre-compiled binaries...

[MiniSat_v1.14_cygwin](#) - Cygwin/Windows binary.

miniSAT example

The screenshot shows a terminal window with the following content:

```
$ miniSAT test.cnf test.ans
===== [MINISAT] =====
LEARN
Clauses Literals Lit/Cli |
=====
0 0 0 % |
=====
67 /sec)
267 /sec)
267 /sec)
0.00 % deleted)

CPU time : 0.015 s

SATISFIABLE

mizuhito@miranda ~/Lecture/Mogi070824
$ more test.ans
SAT
-1 -2 3 0
```

A secondary window titled "MeadowNT.exe@MIRANDA" shows the contents of test.cnf:

```
c *comment lines*
p cnf 3 4
1 2 3 0
-1 2 0
-2 3 0
-1 -2 -3 0
[--]-(Unix)-- test.cnf
```

Red arrows and boxes highlight the following information:

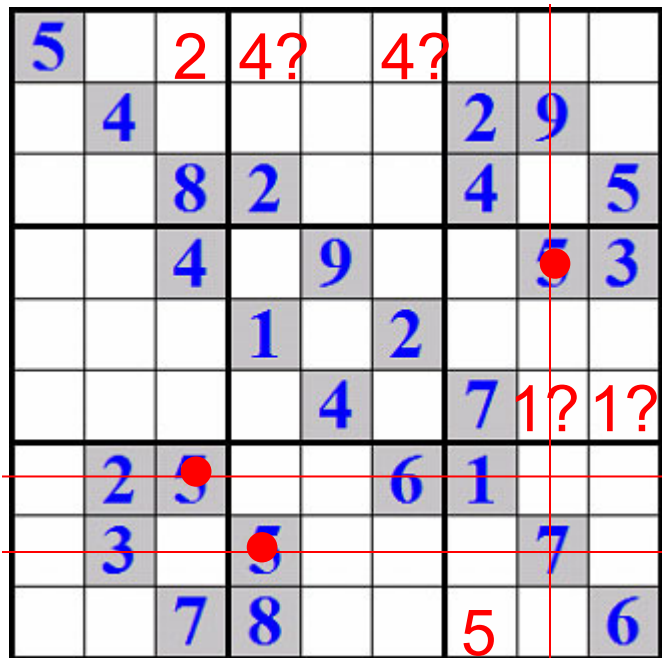
- Number of variables:** 3 (indicated by an arrow pointing to the '3' in the CNF header).
- Number of clauses:** 4 (indicated by an arrow pointing to the '4' in the CNF header).
- Representing:** A box containing the CNF suggested form:
 $x_1 \vee x_2 \vee x_3$
 $\neg x_1 \vee x_2$
 $\neg x_2 \vee x_3$
 $\neg x_1 \vee \neg x_2 \vee \neg x_3$
(0 is the terminator).
- Result:** A box containing the solution:
 $x_1 = 0$
 $x_2 = 0$
 $x_3 = 1.$

The terminal output shows the result is SATISFIABLE and the solution is -1 -2 3 0.

Puzzel: SUDOKU, again

Find placing *each number (1...9) s.t. precisely once in*

- *each row,*
- *each column,*
- *each 3*3 sub-square.*



```
MeadowNT.exe@MIRANDA
Buffers  Files  Tools  Edit  Search  Mule  Help
115
224
272
289
338
342
374
395
434
459
485
493
541
562
654
677
722
735
766
771
823
845
887
937
948
975
996
[--]-(Unix)--  sudokuMar2
```

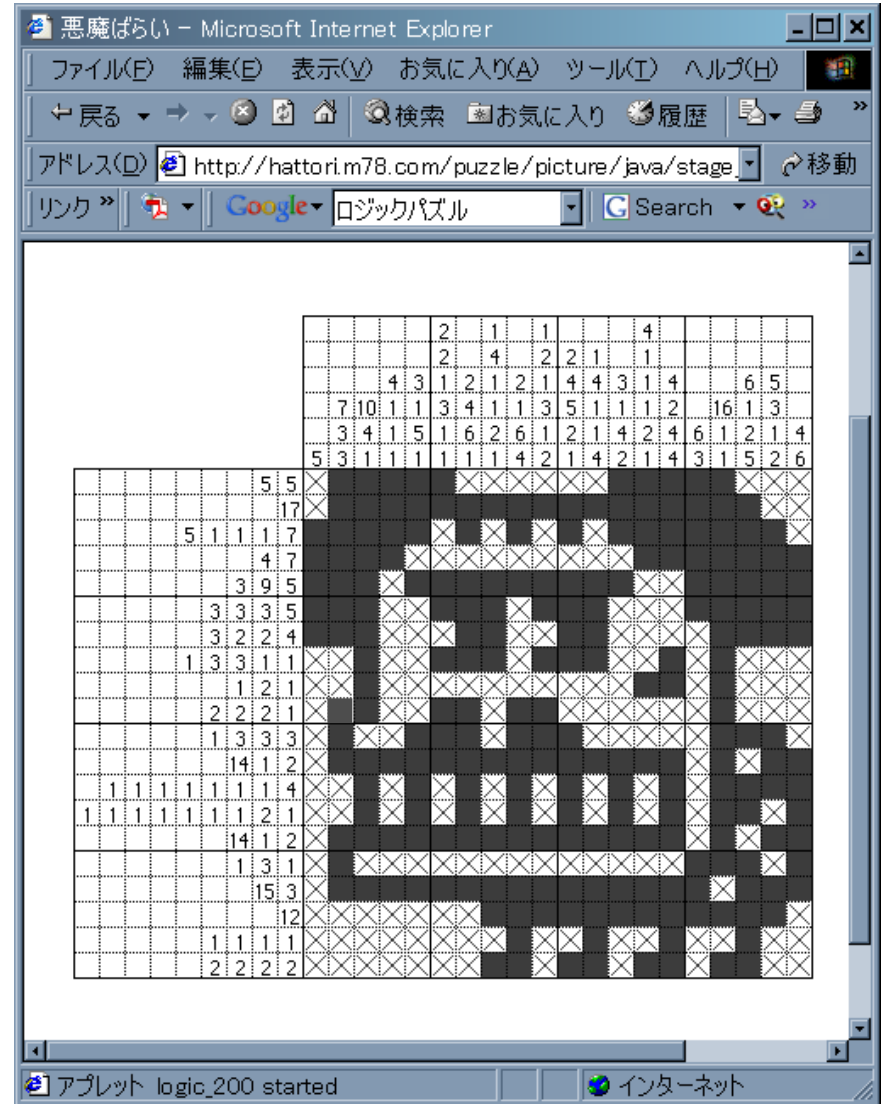
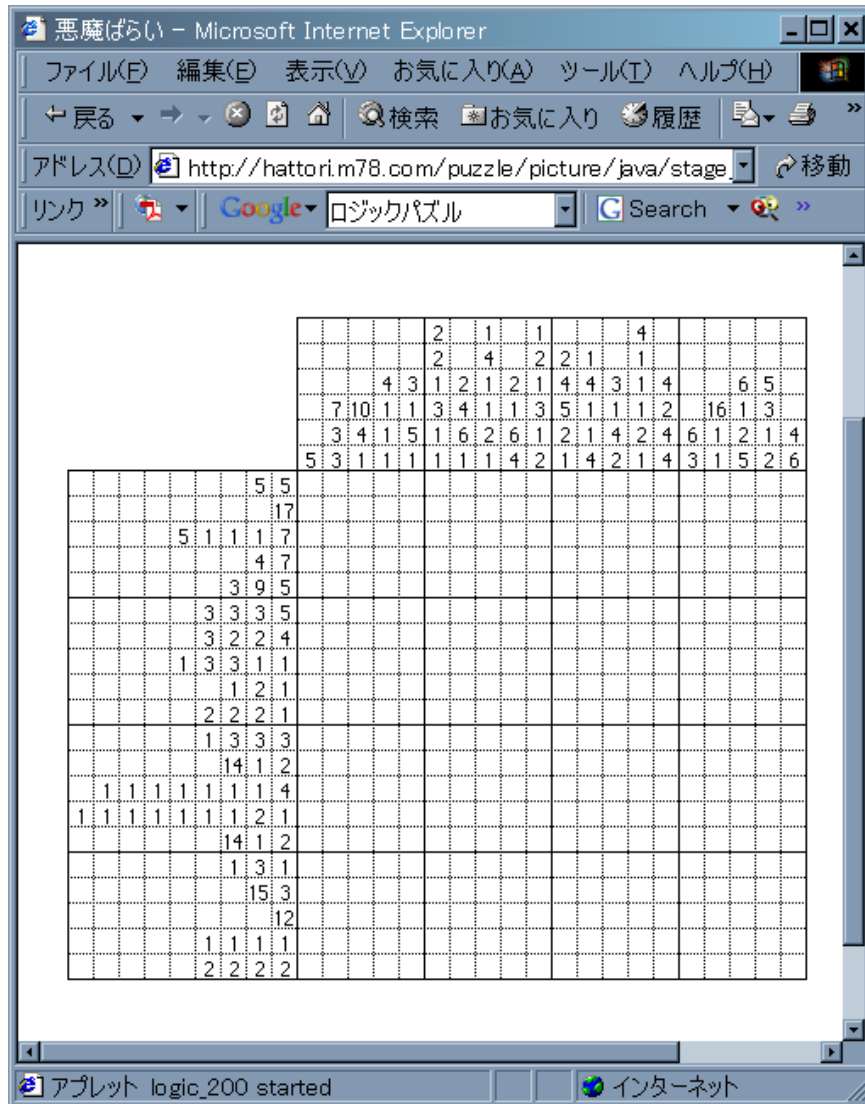
SUDOKU as a SAT problem, again

Lynce, I., Ouakline, J., AI Math 2006

- x_{ijk} iff k at (i,j) where $1 \leq i, j, k \leq 9$
- Each box contains precisely one number.
 $(x_{ij1} \vee x_{ij2} \vee \dots \vee x_{ij9}) \wedge (x_{ij1} \supset \neg x_{ij2})$ (i.e., $\neg x_{ij1} \vee \neg x_{ij2}$), ...
- Precisely once in each row (fix i):
 $(x_{i1k} \vee x_{i2k} \vee \dots \vee x_{i9k}) \wedge (x_{i1k} \supset \neg x_{i2k})$ (i.e., $\neg x_{i1k} \vee \neg x_{i2k}$), ...
- Precisely once in each column (fix j):
 $(x_{1jk} \vee x_{2jk} \vee \dots \vee x_{9jk}) \wedge (x_{1jk} \supset \neg x_{2jk})$ (i.e., $\neg x_{1jk} \vee \neg x_{2jk}$), ...
- Precisely once in each sub-3x3 matrix.
 $(x_{11k} \vee x_{12k} \vee \dots \vee x_{33k}) \wedge (x_{11k} \supset \neg x_{12k})$ (i.e., $\neg x_{11k} \vee \neg x_{12k}$), ...

729vars, approx.8200clauses, in 10-50ms by miniSAT

Logic pictures



Logic picture encoding into SAT

- Assign a variable to each box, and encoding constraints on each column/row

				1	
		3	3	2	1
2	1	x_1	x_2	x_3	x_4
	3	x_6	...		
	5			...	x_{15}

2	1	x_1	x_2	x_3	x_4	x_5
---	---	-------	-------	-------	-------	-------

2	1					
---	---	--	--	--	--	--

2	1					
---	---	--	--	--	--	--

2	1					
---	---	--	--	--	--	--

$$(x_1 \wedge x_2 \wedge \neg x_3 \wedge x_4 \wedge \neg x_5)$$

$$\vee (x_1 \wedge x_2 \wedge \neg x_3 \wedge \neg x_4 \wedge x_5)$$

$$\vee (\neg x_1 \wedge x_2 \wedge x_3 \wedge \neg x_4 \wedge x_5)$$

- DNF needs to be converted into CNF

How to convert DNF into CNF ?

- Distribution law (logically equivalent CNF from DNF)

- ✓ Exponential growth and very inefficient

- $(x_1 \wedge x_2 \wedge \neg x_3 \wedge x_4 \wedge \neg x_5)$

- $\vee (x_1 \wedge x_2 \wedge \neg x_3 \wedge \neg x_4 \wedge x_5)$

- $\vee (\neg x_1 \wedge x_2 \wedge x_3 \wedge \neg x_4 \wedge x_5)$ becomes

- $(x_1 \vee x_1 \vee \neg x_1) \wedge \dots \wedge (x_1 \vee x_1 \vee x_5) \wedge$

- $(x_1 \vee x_2 \vee \neg x_1) \wedge \dots \wedge (x_1 \vee x_2 \vee x_5) \wedge \dots (5^3 \text{ clauses})$

- Tseitin conversion

- ✓ Obtain CNF of equivalent satisfiability with DNF

- ✓ Linear growth (for m-SAT, n-clauses, $\approx (m+1) n$ clauses), e.g., 19 clauses

- ✓ CNF in 3.8MB and solved in 1.8sec by miniSAT.

Tseitin conversion (1970)

- **Ex.** $(x_1 \wedge x_2 \wedge \neg x_3 \wedge x_4 \wedge \neg x_5)$

$\vee(x_1 \wedge x_2 \wedge \neg x_3 \wedge \neg x_4 \wedge x_5) \vee(\neg x_1 \wedge x_2 \wedge x_3 \wedge \neg x_4 \wedge x_5)$ is converted by

✓ Introduce fresh y_1, y_2, y_3 for $x_1 \wedge x_2 \wedge \neg x_3 \wedge x_4 \wedge \neg x_5$,

$x_1 \wedge x_2 \wedge \neg x_3 \wedge \neg x_4 \wedge x_5$, $\neg x_1 \wedge x_2 \wedge x_3 \wedge \neg x_4 \wedge x_5$

✓ $(y_1 \vee y_2 \vee y_3)$ and $y_1 \Leftrightarrow x_1 \wedge x_2 \wedge \neg x_3 \wedge x_4 \wedge \neg x_5$

– $y_1 \supset (x_1 \wedge x_2 \wedge \neg x_3 \wedge x_4 \wedge \neg x_5) \rightarrow$
 $(\neg y_1 \vee x_1) \wedge (\neg y_1 \vee x_2) \wedge (\neg y_1 \vee \neg x_3) \wedge (\neg y_1 \vee x_4) \wedge (\neg y_1 \vee \neg x_5)$

– $(x_1 \wedge x_2 \wedge \neg x_3 \wedge x_4 \wedge \neg x_5) \supset y_1 \rightarrow$
 $\neg x_1 \vee \neg x_2 \vee x_3 \vee \neg x_4 \vee x_5 \vee y_1$

This rule can be ignored (since y_1 to be 0 is crucial)

Tseitin Conversion (Tseitin 1970)

- Logically equivalent conversion may explode exponentially.
- Tseitin conversion is
 - ✓ at most square growth.
 - ✓ recursively adding a new variable as a substitute of each conjunctive subproposition.
- **Th.** Satisfiability is preserved (though validity is **not**).

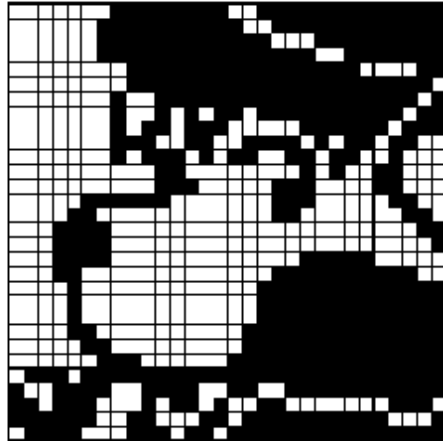
Tseitin conversion

- **Th.** Each proposition A has a CNF A' (square to A in size), such that A is satisfiable if and only if A' is satisfiable.
- **Procedure.** Repeat the following procedure until reaching to a literal. (At each step, the number of literals decreases.)
 - ✓ Step 1: Replace $A \supset B$ with $\neg A \vee B$
 - ✓ Step 2: Push $\neg$ into innermost occurrences by $\neg(A \wedge B) \Leftrightarrow \neg A \vee \neg B$ and $\neg(A \vee B) \Leftrightarrow \neg A \wedge \neg B$.
 - ✓ Step 3: Remove double negation by $\neg \neg A \Leftrightarrow A$
 - ✓ Step 4a: If $A = B \wedge C$ (i.e., there exists $\wedge$ at the top level). Let B' and C' be CNFs with equal satisfiability to B and C , resp.
 - ✓ Step 4b: If $A = B \vee P$ for a propositional variable P . Let $B' = B'_1 \wedge \dots \wedge B'_m$ be a Tseitin encoding of B . Then, Tseitin encoding of A is $(B'_1 \vee P) \wedge \dots \wedge (B'_m \vee P)$.
 - ✓ Step 4c: If $A = B \vee C$ for a conjunctive proposition $C = C_1 \wedge \dots \wedge C_n$. Let B' be a Tseitin encoding of B and P a fresh propositional variable. Then, $B' \vee P$, $\neg P \vee C_1$, ..., $\neg P \vee C_n$, $\neg C_1 \vee \dots \vee \neg C_n \vee P$ are converted separately.

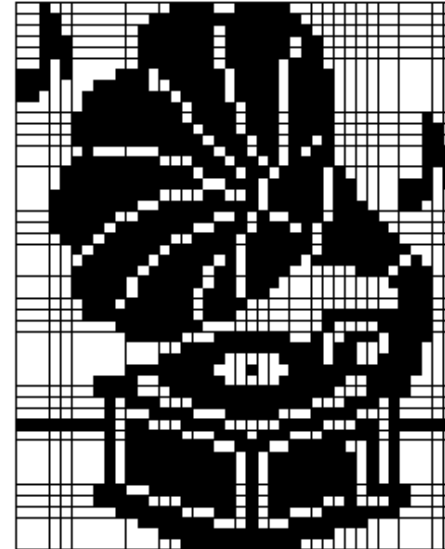
Tseitin conversion still not enough

- Logic pictures larger than 50×40 fail to generate CNF (at least $>1\text{GB}$)

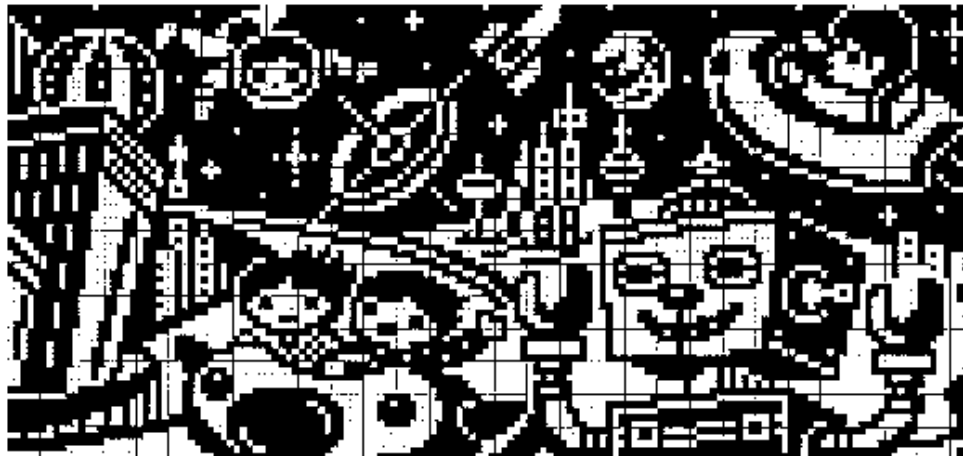
30×30



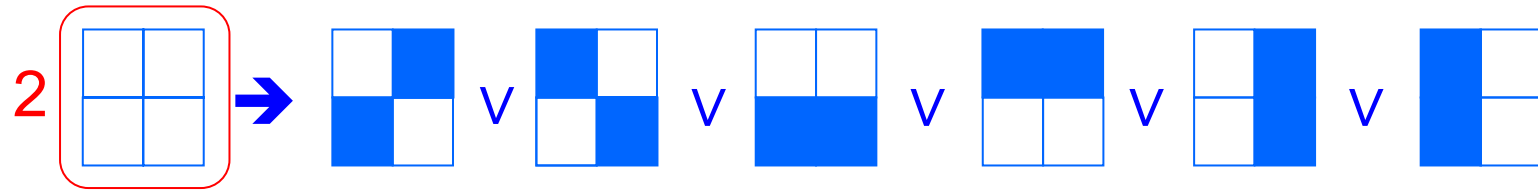
50×40



70×180



Not directly CNF. How to describe in CNF?

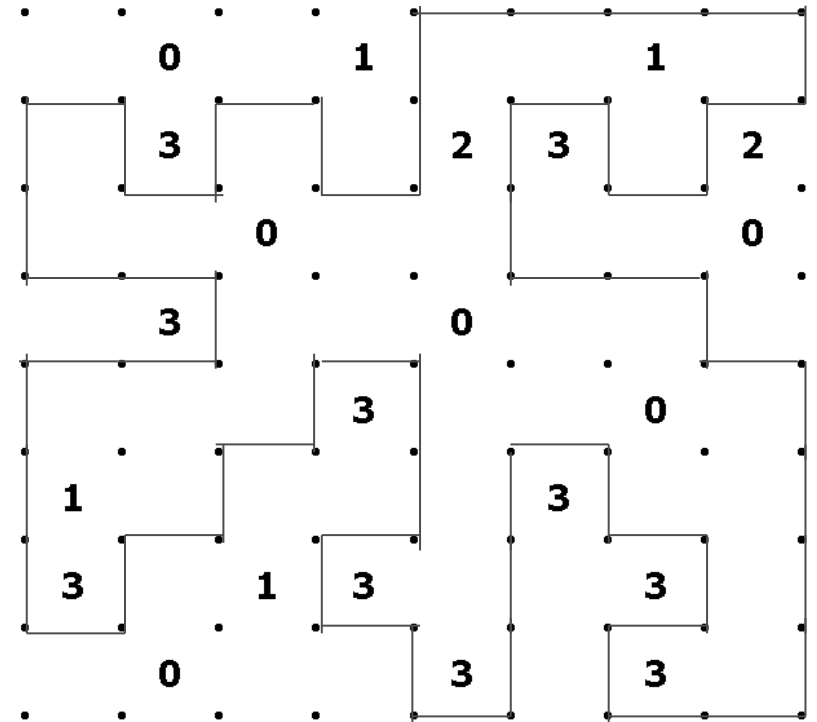


- Exactly 2 variables among 4 variables are true.
 - ✓ $(x \wedge y \wedge \neg z \wedge \neg w) \vee (x \wedge \neg y \wedge z \wedge \neg w) \vee$
 $(\neg x \wedge y \wedge z \wedge \neg w) \vee (x \wedge \neg y \wedge \neg z \wedge w) \vee$
 $(\neg x \wedge y \wedge \neg z \wedge w) \vee (\neg x \wedge \neg y \wedge z \wedge w) : \text{DNF}$
 - ✓ DNF to CNF conversion? ($4^6 = 4096$ clauses)
 - ✓ Tseitin conversion? ($1+6+24 = 31$ clauses)
- Another method
 - ✓ Neither 0, 1, 3, nor 4 variables are true. (10 clauses)

- Not 3 variables
- $\neg((x \vee y \vee z \vee \neg w) \wedge (x \vee y \vee \neg z \vee w) \wedge (x \vee \neg y \vee z \vee w) \wedge (\neg x \vee y \vee z \vee w))$
- $(\neg(x \vee y \vee z \vee \neg w) \vee \neg(x \vee y \vee \neg z \vee w) \vee \neg(x \vee \neg y \vee z \vee w) \vee \neg(\neg x \vee y \vee z \vee w))$
- $((\neg x \wedge \neg y \wedge \neg z \wedge \neg \neg w) \vee \neg(x \vee y \vee \neg z \vee w) \vee \neg(x \vee \neg y \vee z \vee w) \vee \neg(\neg x \vee y \vee z \vee w))$
- CNF ψ –minisat- $\rightarrow x = 1, y = 0, z = 1 \quad x \wedge \neg y \wedge z$
- Not this instance. $\psi \wedge \neg(x \wedge \neg y \wedge z) = \psi \wedge (\neg x \vee \neg \neg y \vee \neg z)$
UNSAT?

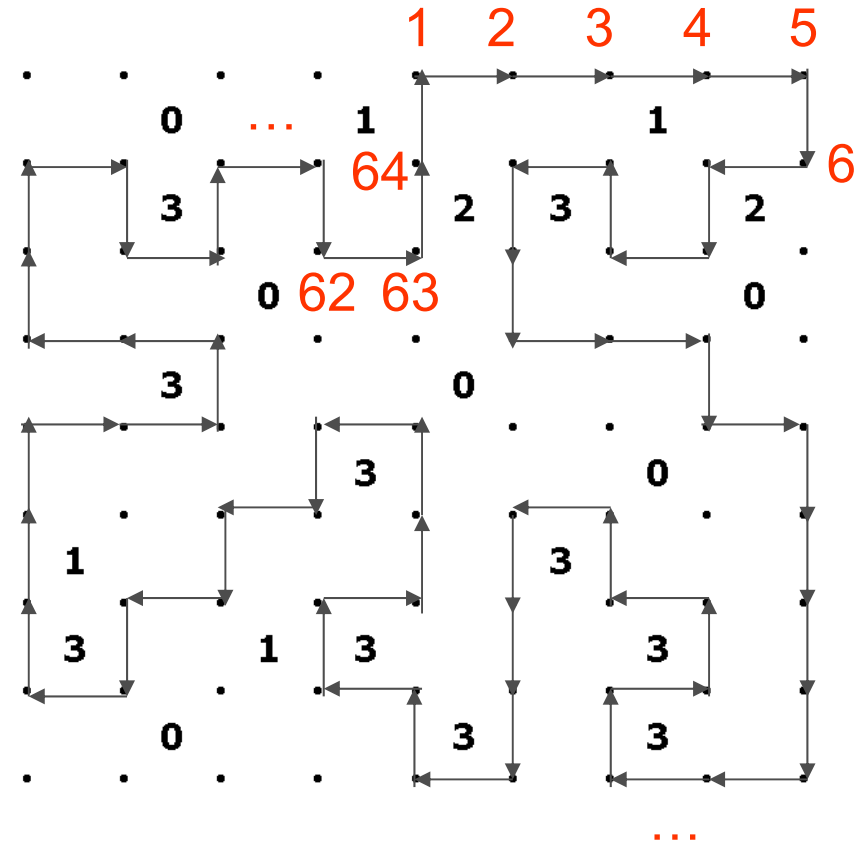
Slitherlink

- Slitherlink
 - ✓ Find *a cycle* such that the numbers of edges of boxes match given numbers.
- Difficulties
 - ✓ How to describe “*Cycle is connected*”.
 - ✓ We consider directed edges.



Different descriptions of a cycle

- X is a (directed) **cycle**
 - ✓ if X is **connected** and each vertex in X has unique successor and ancestor, respectively.
 - ✓ if any **increasing** label (starting from 1) on X has **a unique vertex labeled 1**, and
(Sugar, 神戸大学)



SAT encoding preferences

- In theory, properties on a bounded model can be described by CNF; thus solved by SAT solver.
- Try to describe with $\exists$, but not with $\forall$
 - ✓ $\forall$ will enumerate all possible combinations!
 - ✓ $\exists$ is naturally described as a SAT problem.
- Preferable descriptions
 - ✓ CNF rather than DNF
 - ✓ Fewer variables / clauses
 - ✓ Shorter clauses

Choose appropriate
definition/description
of a target problem